

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 5 LECTURE 2

MACHINE LEVEL PROGRAMMING

Lecture Outline

- ★ History of Intel processors and architectures
- ★ C, assembly, machine code
- ★ Assembly Basics: Programmers Model, Intel x86 Registers, operands
- ★ Data Transfer Instructions
- ★ Arithmetic & logical Instructions
- ★ Intel x86 Assembly Language Programming
- Reading Assignment

MACHINE-LEVEL PROGRAMMING I : BASICS

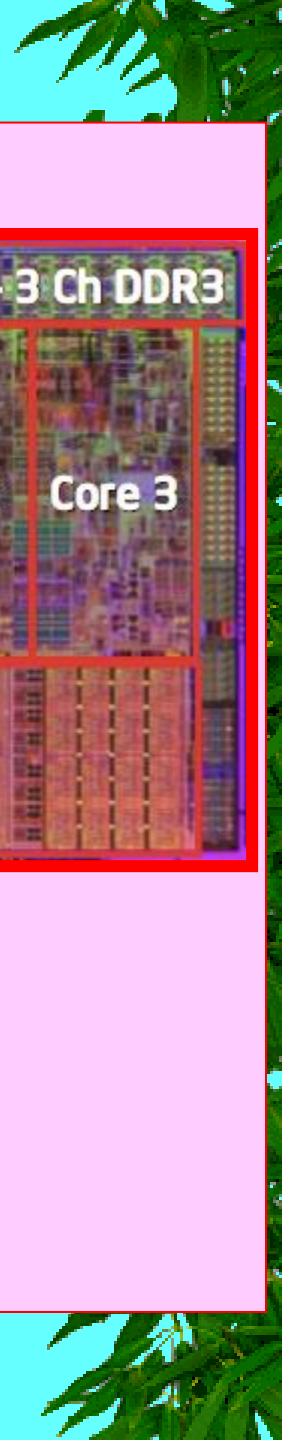
Intel x86 Processors

- ★ **Dominate laptop/desktop/server market**
- ★ **Evolutionary design**
 - Backwards compatible up until 8086, introduced in 1978
 - Added more features as time goes on
- ★ **Complex instruction set computer (CISC)**
 - Many different instructions with many different formats
 - ★ But, only small subset encountered with Linux programs
 - Hard to match performance of Reduced Instruction Set Computers (RISC)
 - But, Intel has done just that!
 - ★ In terms of speed. Less so for low power.

Intel x86 Evolution: Milestones

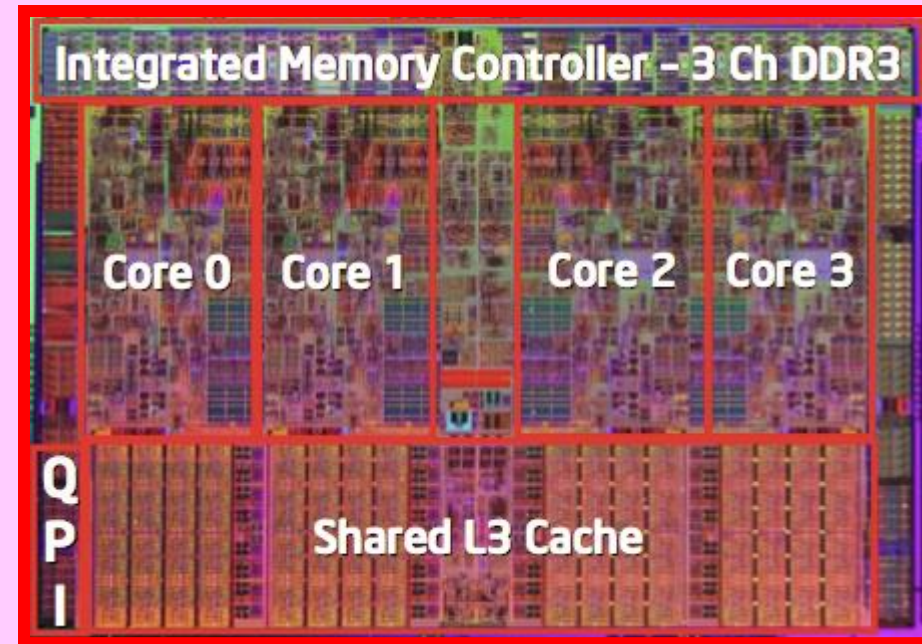
<i>Name</i>	<i>Date</i>	<i>Transistors</i>	<i>MHz</i>
★ 8086	1978	29K	5-10
—	First 16-bit Intel processor. Basis for IBM PC & DOS		
—	1MB address space		
★ 386	1985	275K	16-33
—	First 32 bit Intel processor , referred to as IA32		
—	Added “flat addressing”, capable of running Unix		
★ Pentium 4E	2004	125M	2800-3800
—	First 64-bit Intel x86 processor, referred to as x86-64		
★ Core 2	2006	291M	1060-3500
—	First multi-core Intel processor		
★ Core i7	2008	731M	1700-3900
—	Four cores		

Intel x86 Processors



★ Machine Evolution

—	386	1985	0.3M
—	Pentium	1993	3.1M
—	Pentium/MMX	1997	4.5M
—	PentiumPro	1995	6.5M
—	Pentium III	1999	8.2M
—	Pentium 4	2001	42M
—	Core 2 Duo	2006	291M
—	Core i7	2008	731M



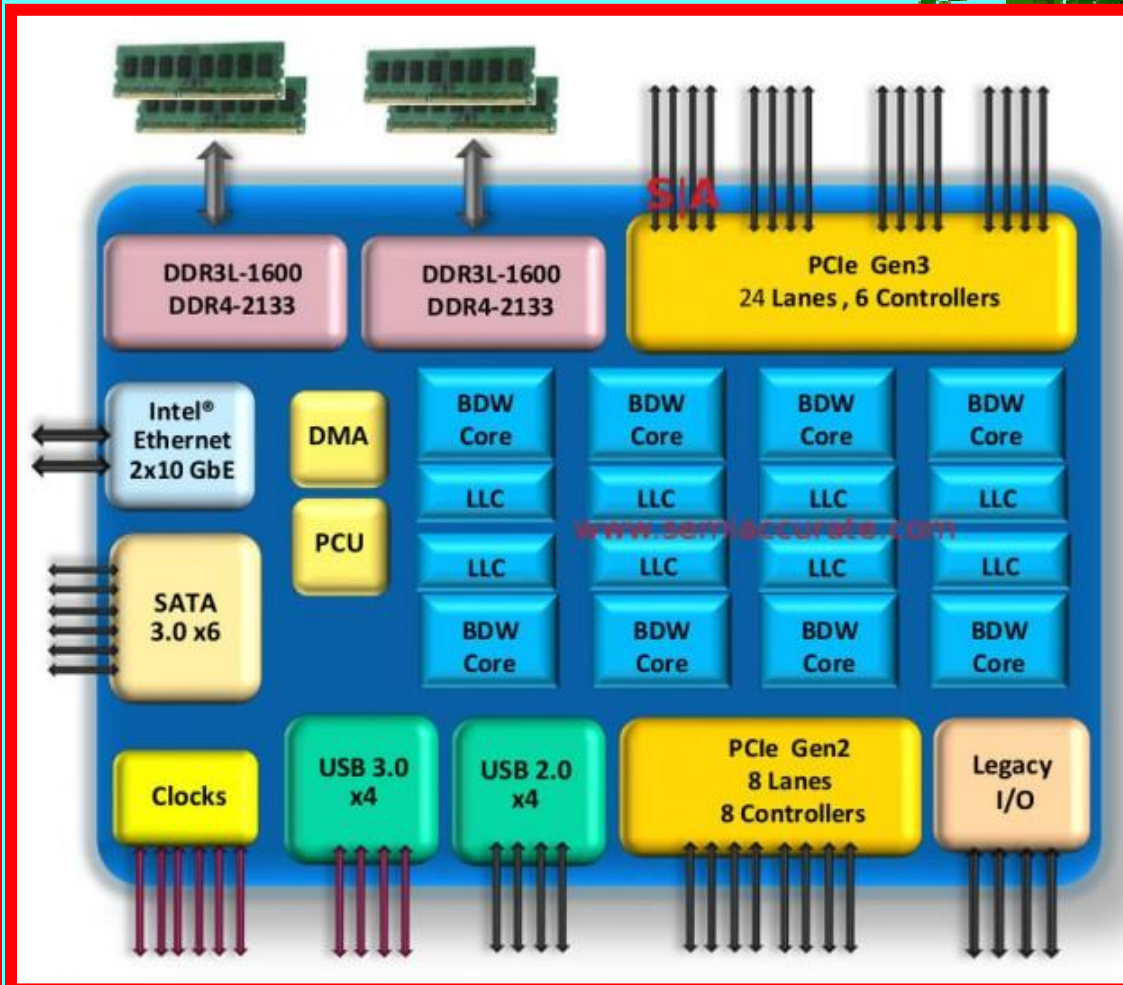
★ Added Features

- Instructions to support multimedia operations
- Instructions to enable more efficient conditional operations
- Transition from 32 bits to 64 bits
- More cores

2015 State of the Art

Core i7 Broadwell 2015

- ★ Desktop Model
 - 4 cores
 - Integrated graphics
 - 3.3-3.8 GHz
 - 65W
- ★ Server Model
 - 8 cores
 - Integrated I/O
 - 2-2.6 GHz
 - 45W



BDW – BroadWell MICROARCHITECTURE based on Intel 14nm process
LLC – Last Level Cache

x86 Clones: Advanced Micro Devices (AMD)

- ★ **Historically**

- AMD has followed just behind Intel
- A little bit slower, a lot cheaper

- ★ **Then**

- Recruited top circuit designers from Digital Equipment Corp. and other downward trending companies
- Built Opteron: tough competitor to Pentium 4
- Developed x86-64, their own extension to 64 bits

- ★ **Recent Years**

- Intel got its act together

- ★ **Leads the world in semiconductor technology**

- AMD has fallen behind

- ★ **Relies on external semiconductor manufacturer**

Intel's 64-Bit History

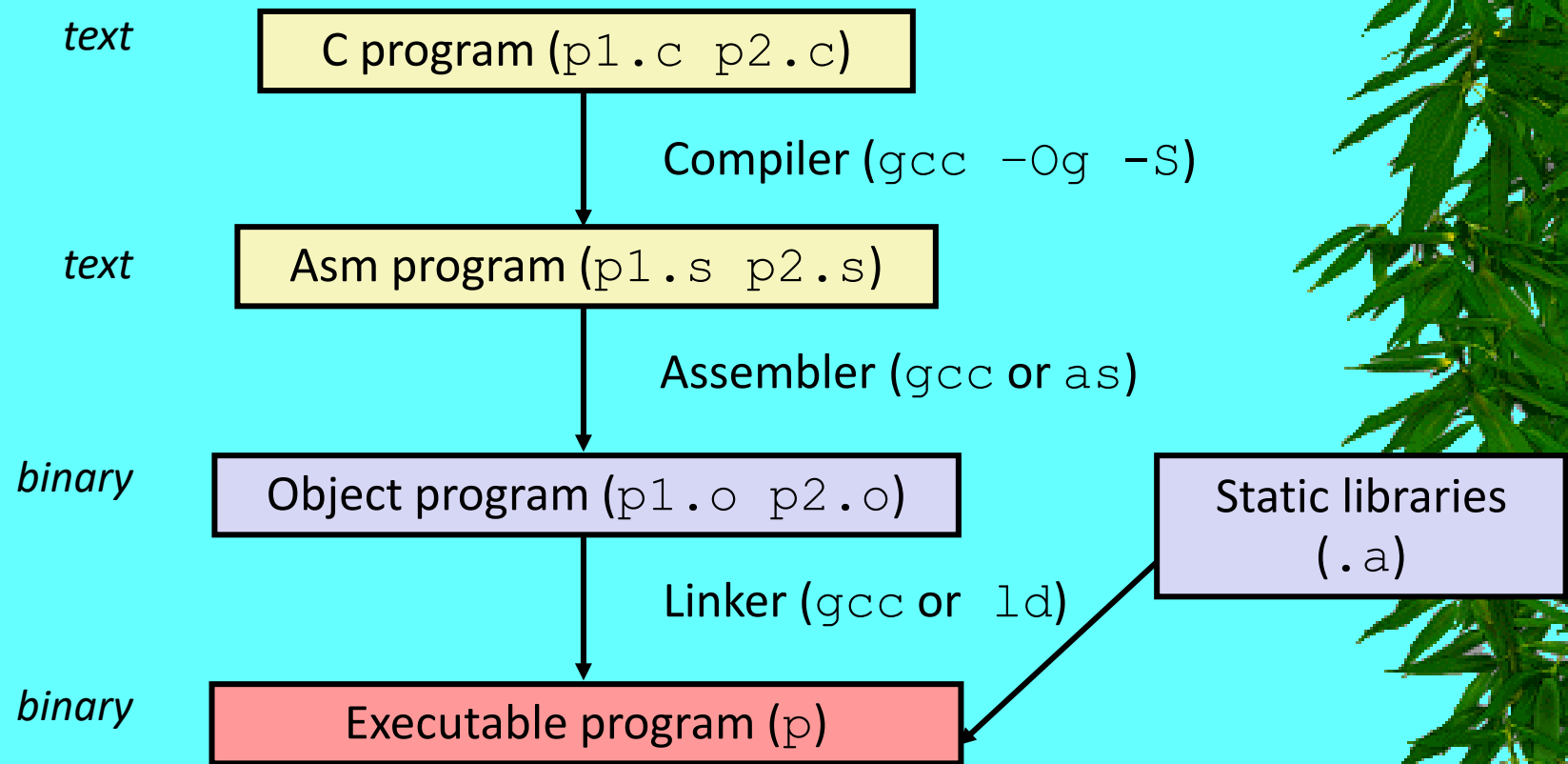
- ★ **2001: Intel Attempts Radical Shift from IA32 to IA64**
 - Totally different architecture (Itanium)
 - Executes IA32 code only as legacy
 - Performance disappointing
- ★ **2003: AMD Steps in with Evolutionary Solution**
 - x86-64 (now called “AMD64”)
- ★ **Intel Felt Obligated to Focus on IA64**
 - Hard to admit mistake or that AMD is better
- ★ **2004: Intel Announces EM64T extension to IA32**
 - Extended Memory 64-bit Technology
 - Almost identical to x86-64!
- ★ **All but low-end x86 processors support x86-64**
 - But, lots of code still runs in 32-bit mode

Definitions

- * **Architecture: (also ISA: Instruction Set architecture)** The parts of a processor design that one needs to understand to write assembly/machine code.
 - Examples: instruction set specification, registers.
- * **Microarchitecture:** Implementation of the architecture.
 - Examples: cache sizes and core frequency.
- * **Code Forms:**
 - **Machine Code:** The byte-level programs that a processor executes
 - **Assembly Code:** A text representation of machine code
- * **Example ISAs:**
 - Intel: x86, IA32, Itanium, x86-64
 - ARM: Used in almost all mobile phones

Turning C into Object Code

- Code in files `p1.c p2.c`
- Compile with command: `gcc -Og p1.c p2.c -o p`
- ★ Use basic optimizations (`-Og`) [New to recent versions of GCC]
- ★ Put resulting binary in file `p`



Compiling Into Assembly

C Code (sum.c)

```
long plus(long x, long y);  
  
void sumstore(long x, long y,  
              long *dest)  
{  
    long t = plus(x, y);  
    *dest = t;  
}
```

Generated x86-64 Assembly

```
sumstore:  
    pushq    %rbx  
    movq     %rdx, %rbx  
    call     plus  
    movq     %rax, (%rbx)  
    popq     %rbx  
    ret
```

Obtain (on your machine) with command

```
gcc -Og -S sum.c
```

Produces file `sum.s`

Warning: Will get very different results on non- (Andrew Linux, Mac OS-X, ...) due to different versions of gcc and different compiler settings.

Compiling Into Assembly

```
/* Program mstore.c */
```

```
long mult2(long, long);
```

```
void multstore(long x, long y, long *dest) {  
    long t = mult2(x, y);  
    *dest = t;  
}
```

```
$gcc -Og -S mstore.c
```

```
$gcc -Og -S mstore.c
```

```
$cat mstore.s
```

```
/* Assembly file mstore.s */
```

```
.file      "mstore.c"  
.text  
.globl    multstore  
.type     multstore, @function multstore:  
  
.LFB0:  
  
.cfi_startproc  
pushq    %rbx  
movq     %rdx, %rbx  
call     mult2  
movq     %rax, (%rbx)  
popq     %rbx  
ret  
.cfi_endproc  
  
.LFE0:  
  
.size     multstore, .-multstore  
.ident    "GCC: (Ubuntu 5.4.0-6ubuntu1~16.04.2) 5.4.0 20160609"  
.section  .note.GNU-stack,"",@progbits
```

Compiling Into Assembly

```
/* Program mstore.c */
```

```
long mult2(long, long);
```

```
void multstore(long x, long y, long *dest) {  
    long t = mult2(x, y);  
    *dest = t;  
}
```

```
$gcc -Og -c mstore.c
```

```
$objdump -d mstore.o
```

```
$gcc -Og -c mstore.c
```

```
$objdump -d mstore.o
```

```
mstore.o: file format elf64-x86-64
```

Disassembly of section .text:

```
0000000000000000 <multstore>:
```

0:	53	push	%rbx
1:	48 89 d3	mov	%rdx,%rbx
4:	e8 00 00 00 00	callq	9 <multstore+0x9>
9:	48 89 03	mov	%rax,(%rbx)
c:	5b	pop	%rbx
d:	c3	retq	

Compiling Into Assembly

```
/* Program main.c */
#include <stdio.h>
void multstore(long, long, long *);
int main() {
    long d;
    multstore(2, 3, &d);
    printf("2 * 3 --> %ld\n", d);
    return 0;
}

long mult2(long a, long b) {
    long s = a * b;
    return s;
}
```

```
/* Program mstore.c */

long mult2(long, long);

void multstore(long x, long y, long *dest) {
    long t = mult2(x, y);
    *dest = t;
}
```

```
$gcc -Og -o prog main.c mstore.c
$objdump -d prog
```

```
$gcc -Og -S main.c
$cat main.s
.file             "main.c"
.section          .rodata.str1.1,"aMS",@progbits,1
.LC0:
.string          "2 * 3 --> %ld\n"
.text
.globl           main
.type            main, @function

main:
.LFB23:
.cfi_startproc
subq             $24, %rsp
.cfi_def_cfa_offset 32
movq             %fs:40, %rax
movq             %rax, 8(%rsp)
xorl             %eax, %eax
movq             %rsp, %rdx
movl             $3, %esi
movl             $2, %edi
call             multstore
movq             (%rsp), %rdx
movl             $.LC0, %esi
movl             $1, %edi
movl             $0, %eax
call             __printf_chk
movq             8(%rsp), %rcx
xorq             %fs:40, %rcx
je               .L2
call             __stack_chk_fail

.L2:
movl             $0, %eax
addq             $24, %rsp
.cfi_def_cfa_offset 8
ret
.cfi_endproc

.LFE23:
.size            main, .-main
.globl           mult2
.type            mult2, @function

mult2:
.LFB24:
.cfi_startproc
movq             %rdi, %rax
imulq            %rsi, %rax
ret
.cfi_endproc

.LFE24:
.size            mult2, .-mult2
.ident           "GCC: (Ubuntu 5.4.0-6ubuntu1~16.04.2) 5.4.0 20160609"
.section         .note.GNU-stack,"",@progbits
```


Object Code

Code for sumstore

0x0400595:

0x53

0x48

0x89

0xd3

0xe8

0xf2

0xff

0xff

0xff

0x48

0x89

0x03

0x5b

0xc3

Total of 14 bytes

**Each instruction
1, 3, or 5 bytes**

**Starts at address
0x0400595**

- * Assembler
 - Translates `.s` into `.o`
 - Binary encoding of each instruction
 - Nearly-complete image of executable code
 - Missing linkages between code in different files
- * Linker
 - Resolves references between files
 - Combines with static run-time libraries
 - * E.g., code for `malloc`, `printf`
 - Some libraries are *dynamically linked*
 - * Linking occurs when program begins execution

Machine Instruction Example

```
*dest = t;
```

```
movq %rax, (%rbx)
```

```
0x40059e: 48 89 03
```

- ★ **C Code**

- Store value `t` where designated by `dest`

- ★ **Assembly**

- Move 8-byte value to memory
- Quad words in x86-64 parlance

- Operands:

`t:` Register `%rax`

`dest:` Register `%rbx`

`*dest:` Memory `M[%rbx]`

- ★ **Object Code**

- 3-byte instruction
- Stored at address `0x40059e`

Disassembling Object Code

Disassembled

```
0000000000400595 <sumstore>:
 400595: 53                push    %rbx
 400596: 48 89 d3          mov     %rdx,%rbx
 400599: e8 f2 ff ff ff    callq   400590 <plus>
 40059e: 48 89 03          mov     %rax, (%rbx)
 4005a1: 5b                pop     %rbx
 4005a2: c3                retq
```

* Disassembler

objdump -d sum

- Useful tool for examining object code
- Analyzes bit pattern of series of instructions
- Produces approximate rendition of assembly code
- Can be run on either a .out (complete executable) or .o file

Alternate Disassembly

Disassembled

Object

0x0400595:

0x53

0x48

0x89

0xd3

0xe8

0xf2

0xff

0xff

0xff

0x48

0x89

0x03

0x5b

0xc3

Dump of assembler code for function sumstore:

```
0x0000000000400595 <+0>: push    %rbx
0x0000000000400596 <+1>: mov     %rdx,%rbx
0x0000000000400599 <+4>: callq  0x400590 <plus>
0x000000000040059e <+9>: mov     %rax, (%rbx)
0x00000000004005a1 <+12>: pop     %rbx
0x00000000004005a2 <+13>: retq
```

* Within gdb Debugger

`gdb sum`

`disassemble sumstore`

– `Disassemble procedure`

`sumstore`

– `Examine the 14 bytes starting at sumstore`

What Can be Disassembled?

```
% objdump -d WINWORD.EXE
```

```
WINWORD.EXE:      file format pei-i386
```

```
No symbols in "WINWORD.EXE".
```

```
Disassembly of section .text:
```

```
30001000 <.text>:
```

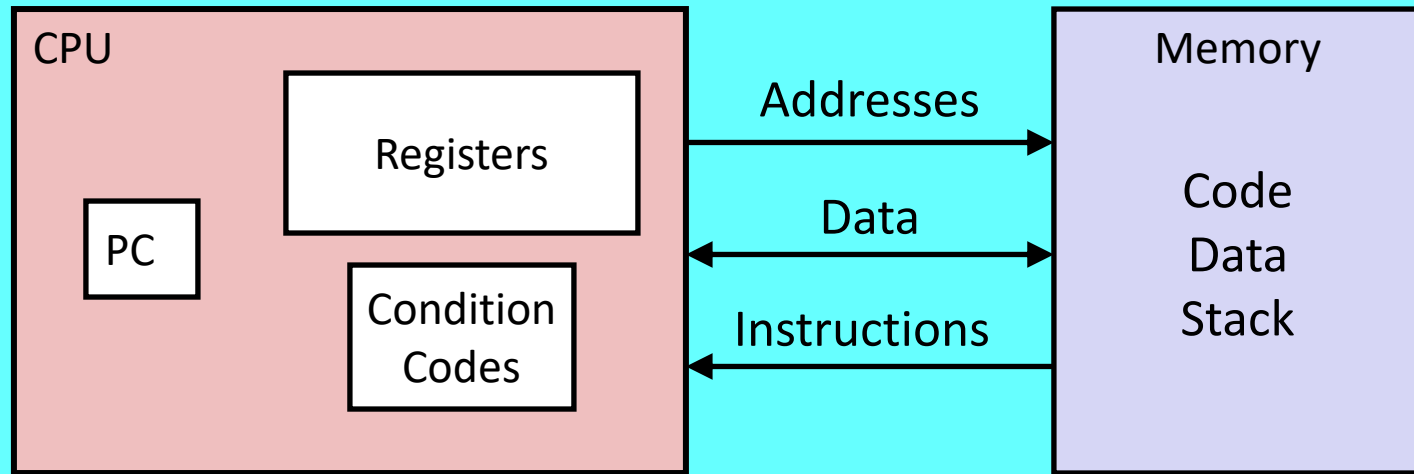
```
30001000:  55                push    %ebp
30001001:  8b ec            mov     %esp,%ebp
30001003:  6a ff            push    $0xffffffff
30001005:  68 90 10 00 30  push    $0x30001090
3000100a:  68 91 dc 4c 30  push    $0x304cdc91
```

Reverse engineering forbidden by
Microsoft End User License Agreement

- * Anything that can be interpreted as executable code
- * Disassembler examines bytes and reconstructs assembly source

MACHINE-LEVEL PROGRAMMING I: PROGRAMMER'S MODEL

Assembly/Machine Code View



Programmer-Visible State

PC: Program counter

- * Address of next instruction
- * Called “RIP” (x86-64)

Register file

- * Heavily used program data

Condition codes

- * Store status information about most recent arithmetic or logical operation
- * Used for conditional branching

Memory

- * Byte addressable array
- * Code and user data
- * Stack to support procedures

Assembly Characteristics: Data Types

- ★ “Integer” data of 1, 2, 4, or 8 bytes
 - Data values
 - Addresses (untyped pointers)
- ★ Floating point data of 4, 8, or 10 bytes
- ★ Code: Byte sequences encoding series of instructions
- ★ No aggregate types such as arrays or structures
 - Just contiguously allocated bytes in memory

Assembly Characteristics: Operations

- ★ Perform arithmetic function on register or memory data
- ★ Transfer data between memory and register
 - Load data from memory into register
 - Store register data into memory
- ★ Transfer control
 - Unconditional jumps to/from procedures
 - Conditional branches

x86-64 Integer Registers

%rax	%eax
%rbx	%ebx
%rcx	%ecx
%rdx	%edx
%rsi	%esi
%rdi	%edi
%rsp	%esp
%rbp	%ebp

%r8	%r8d
%r9	%r9d
%r10	%r10d
%r11	%r11d
%r12	%r12d
%r13	%r13d
%r14	%r14d
%r15	%r15d

- Can reference low-order 4 bytes (also low-order 1 & 2 bytes)

Some History: IA32 Registers

Origin (mostly obsolete)

general purpose

%eax	%ax	%ah	%al
%ecx	%cx	%ch	%cl
%edx	%dx	%dh	%dl
%ebx	%bx	%bh	%bl
%esi	%si		
%edi	%di		
%esp	%sp		
%ebp	%bp		

accumulate

counter

data

base

Source index

Destination index

Stack pointer

Base pointer

suffix

16-bit virtual registers (backwards compatibility)

63 32 31 16 15 8 7 0

%rax

%eax

%ax

%ah

%al

Moving Data

* Moving Data

`movq Source, Dest` (Use AT&T Terminology)

* Operand Types

- **Immediate:** Constant integer data
 - * Example: `$0x400`, `$-533`
 - * Like C constant, but prefixed with ``$'`
 - * Encoded with 1, 2, 4 or 8 bytes
- **Register:** One of 16 integer registers
 - * Example: `%rax`, `%r13`
 - * But `%rsp` reserved for special use
 - * Others have special uses for particular instructions
- **Memory:** 8 consecutive bytes of memory at address given by register
 - * Simplest example: `(%rax)`
 - * Various other “address modes”

`%rax`

`%rcx`

`%rdx`

`%rbx`

`%rsi`

`%rdi`

`%rsp`

`%rbp`

`%rN`

Where N = 815

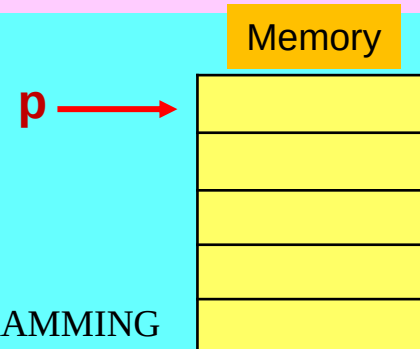
movq Operand Combinations

	Source	Dest	Src, Dest	C Analog
movq	Imm	Reg	movq \$0x4, %rax	temp = 0x4;
		Mem	movq \$-147, (%rdx)	*p = -147;
	Reg	Reg	movq %rbx, %rcx	temp2 = temp1;
		Mem	movq %rax, (%rdx)	*p = temp;
	Mem	Reg	movq (%rdx), %rax	temp = *p;
		Mem		

Cannot do memory-memory transfer with a single instruction

REGISTER MAPPING :

%rax ← temp
%rbx ← temp1
%rcx ← temp2
%rdx ← p



Example of Simple Addressing Modes

C Code

```
void swap
(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

Assembly Code

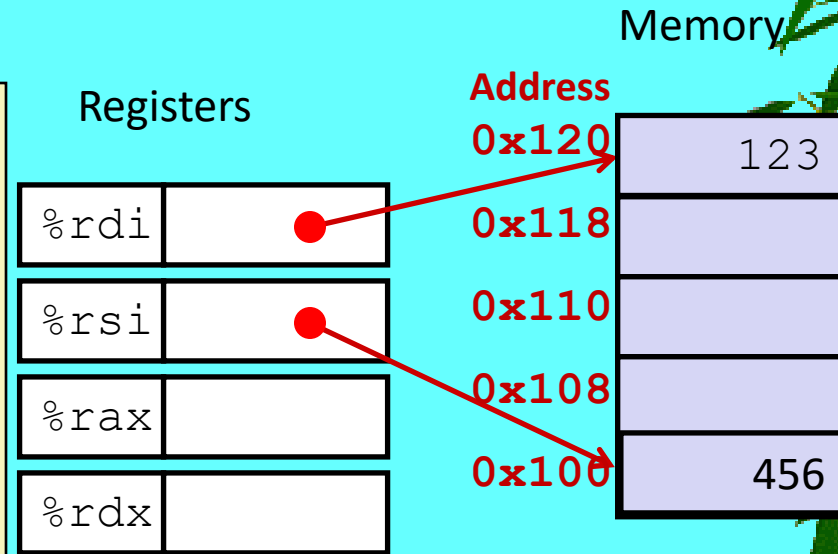
```
swap:
    movq    (%rdi), %rax
    movq    (%rsi), %rdx
    movq    %rdx, (%rdi)
    movq    %rax, (%rsi)
    ret
```

REGISTER MAPPING :

```
%rdi ← xp    %rsi ← yp
%rax ← t0    %rdx ← t1
```

Understanding Swap()

```
void swap
(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```



Register Value

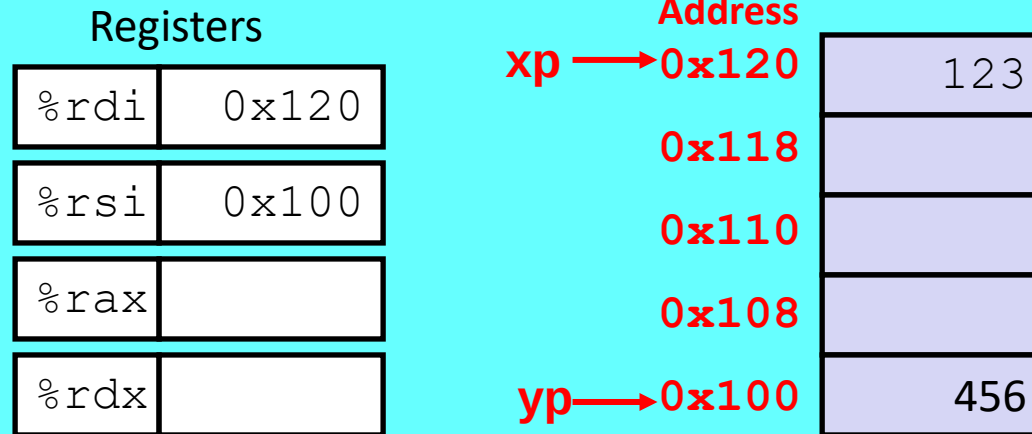
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

Understanding `Swap()`

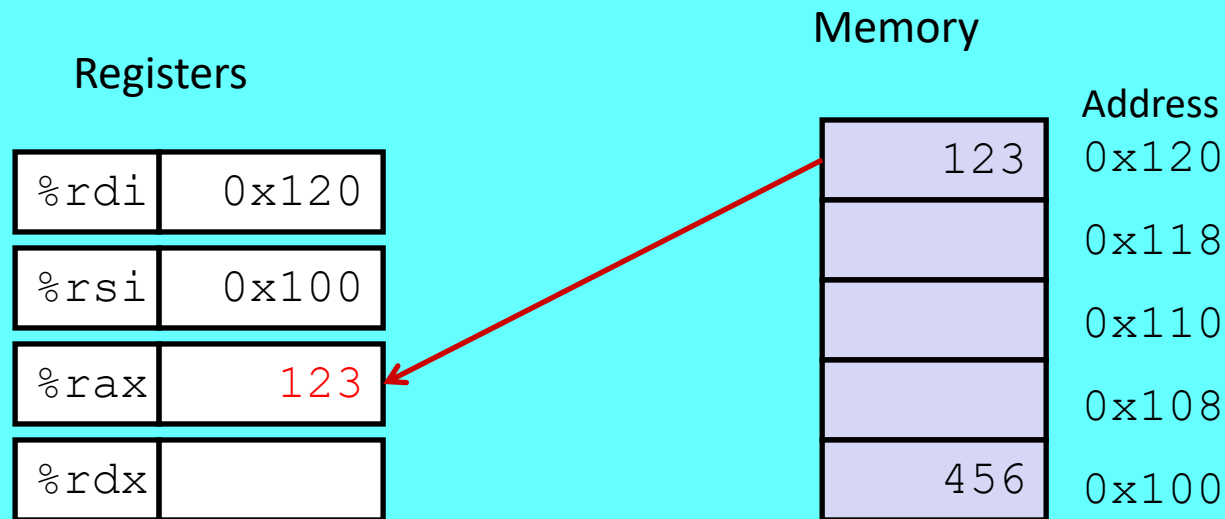
```
movq $xp, %rdi
movq $yp, %rsi
call swap
```



swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

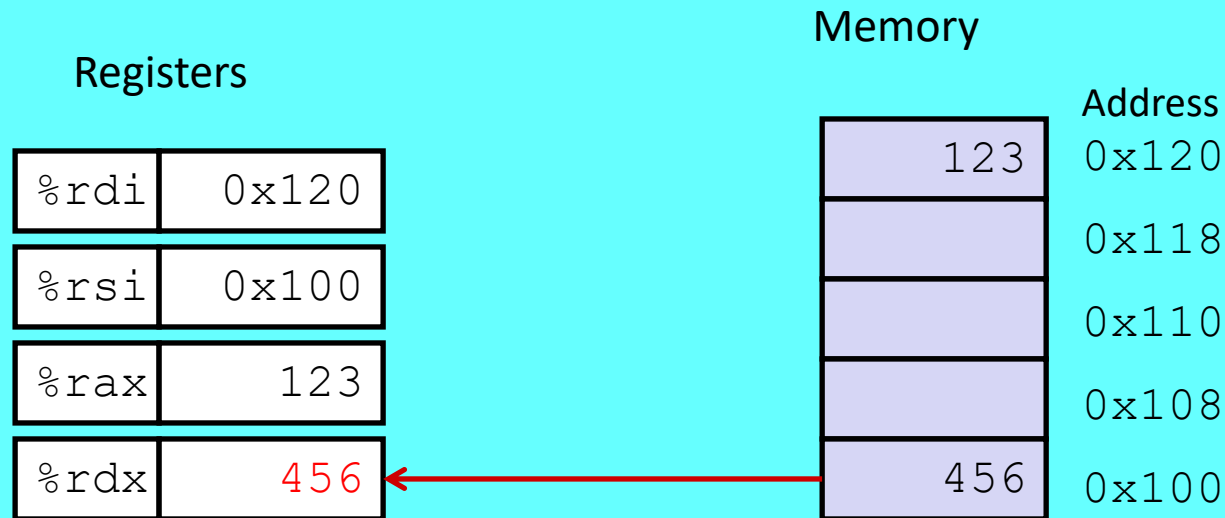

Understanding `Swap()`



`swap:`

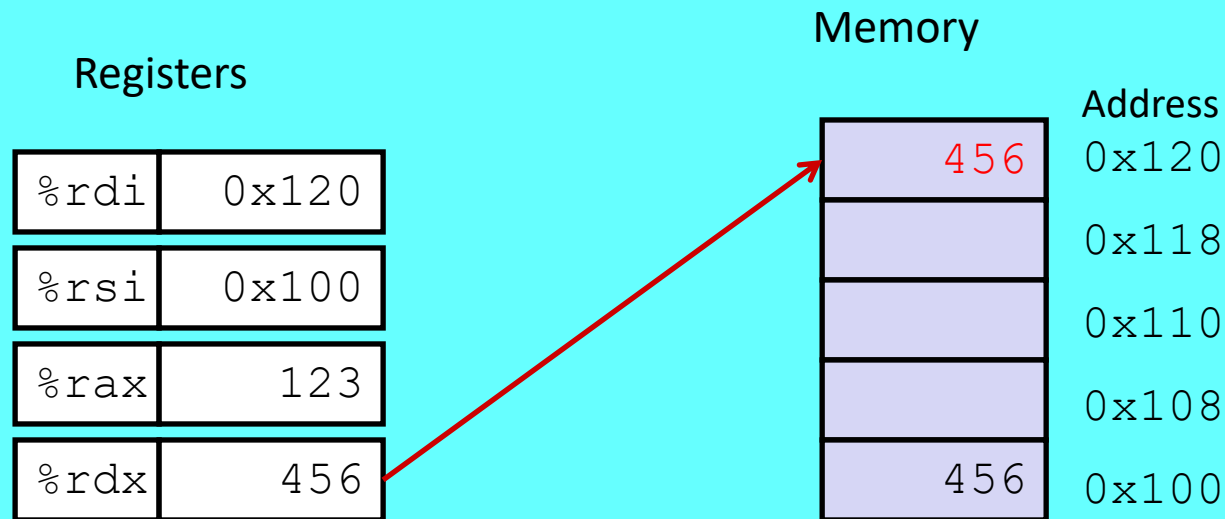
```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

Understanding `Swap()`



```
swap:
    movq    (%rdi), %rax    # t0 = *xp
    movq    (%rsi), %rdx    # t1 = *yp
    movq    %rdx, (%rdi)    # *xp = t1
    movq    %rax, (%rsi)    # *yp = t0
    ret
```

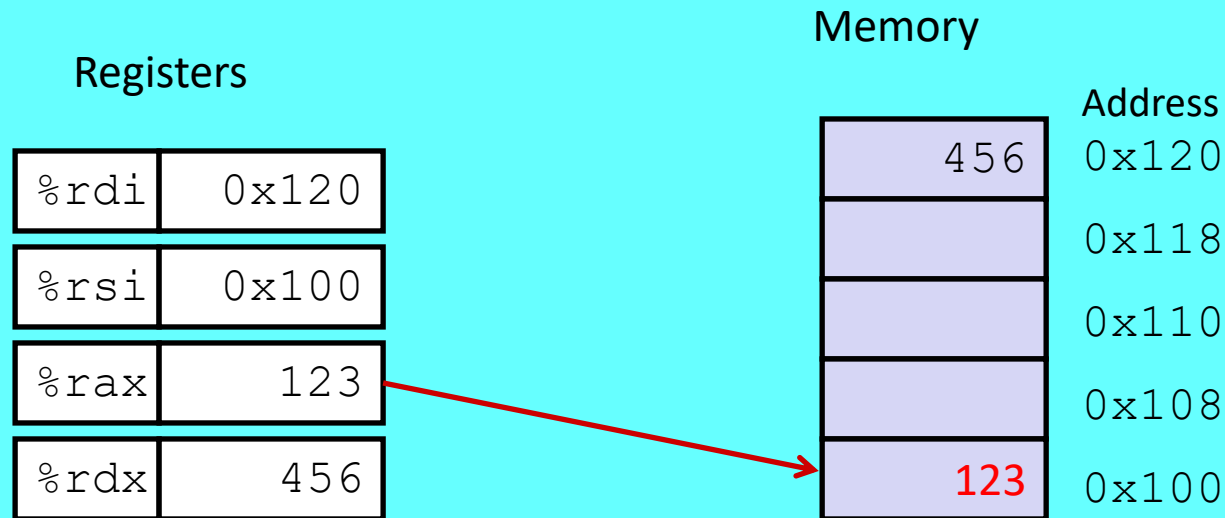
Understanding `Swap()`



`swap:`

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

Understanding `Swap()`



`swap:`

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

Addressing Modes

★ Immediate Addressing Mode

- Constant data is specified immediately in the instruction itself.

movq \$123, %rax

movq \$0xabcd1234, %rax

movq \$-4123, %rax

★ Register addressing mode Reg[R]

- Register R specifies the data.

movq %rax, %rbx

Simple Memory Addressing Modes

★ Normal (R) $\text{Mem}[\text{Reg}[R]]$

- Register R specifies memory address
- Pointer dereferencing in C

```
movq %rax, (%rsi)
```

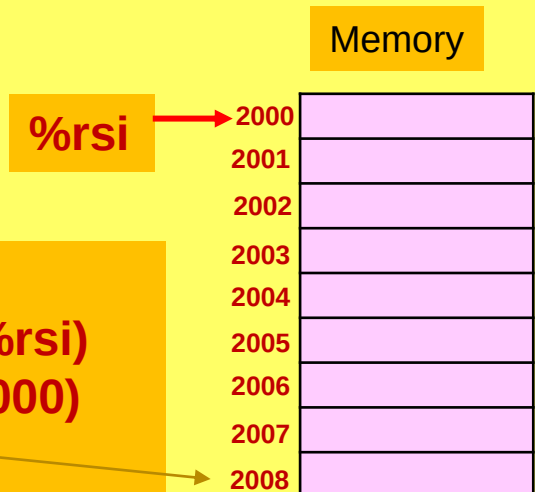
```
movq (%rsi), %rbx
```

★ Displacement D(R) $\text{Mem}[\text{Reg}[R]+D]$

- Register R specifies start of memory region
- Constant displacement D specifies offset

```
movq 8(%rsi), %rdx
```

$\%rsi = 2000$
 $8(\%rsi) = (8 + \%rsi)$
 $= (8 + 2000)$
 $= (2008)$



Simple Memory Addressing Modes

Example :

```
movq $0xabcdef68,%rax
```

%rax = ffffffffabcdef68

```
movq $list,%rsi
```

**list = 2000 address
%rsi = 2000**

```
movq %rax, (%rsi)
```

%rsi → 2000

```
movq (%rsi),%rbx
```

%rbx = ffffffffabcdef68

Base address

%rsi

list: → 2000

Memory

2000
2001
2002
2003
2004
2005
2006
2007
2008

Memory

2000
2001
2002
2003
2004
2005
2006
2007
2008

68

ef

cd

ab

ff

ff

ff

ff

Little-endian byte ordering

Complete Memory Addressing Modes

- ★ Most General Form – Memory addressing with displacement, base and indexing

$D(Rb, Ri, S) \text{ Mem}[\text{Reg}[Rb] + S * \text{Reg}[Ri] + D]$

- D: Constant “displacement” 1, 2, or 4 bytes
- Rb: Base register: Any of 16 integer registers
- Ri: Index register: Any, except for `%rsp`
- S: Scale: 1, 2, 4, or 8 (*why these numbers?*)

- ★ **Special Cases**

Base Register

Index Register

(Rb, Ri)

$\text{Mem}[\text{Reg}[Rb] + \text{Reg}[Ri]]$

$D(Rb, Ri)$

$\text{Mem}[\text{Reg}[Rb] + \text{Reg}[Ri] + D]$

(Rb, Ri, S)

$\text{Mem}[\text{Reg}[Rb] + S * \text{Reg}[Ri]]$

Scale factor

`long int list[100];`
`long int a = list[5];`

Index

Base address

Memory Addressing Mode with Base & Indexing

Example :

```
int list[100];
```

```
a = list[5];
```

Base address = list

Index = 5

Scale factor S = size of each element
= sizeof(int) = 4 bytes

Effective Address = EA
= Base address + index * S

REGISTER MAPPING:

Base Register : `%rdi` $\leftarrow$ list = 0x2000

Index Register : `%rsi` = 5, S=4, `%eax` $\leftarrow$ a

```
movq $list,%rdi
```

```
movq $5, %rsi
```

```
movl (%rdi, %rsi, 4), %eax
```

EA = `%rdi` + `%rsi` * 4 = 0x2000 + 5 * 4
= 0x2000 + 0x14 = 0x2014

(0x2014) $\rightarrow$ `%eax` i.e, 4 bytes from memory

locations starting from 0x2014 are moved to `%eax`

`%rdi`

list: $\rightarrow$

list = 2000 address
`%rdi` = 2000

Memory

2000	list[0]
2001	
2002	
2003	
2004	list[1]
2005	
2006	
2007	
2008	list[2]
2009	
200A	
200B	
200C	list[3]
200D	
200E	
200F	
2010	list[4]
2011	
2012	
2013	
2014	list[5]
2015	
2016	
2017	
2018	list[6]
2019	
201A	

Address Computation Examples

<code>%rdx</code>	<code>0xf000</code>
<code>%rcx</code>	<code>0x0100</code>

Base Address

index

`a = list[100];`

list:

f000
f001
f002
f003
f004
f005
f006
f007
f008

```
movq 0x8(%rdx), %rbx
movq (%rdx, %rcx), %rbx
movq (%rdx, %rcx, 4), %rbx
movq 0x80(%rdx, %rcx, 2), %rbx
```

Expression	Address Computation	Address
<code>0x8(%rdx)</code>	<code>0x8 + 0xf000</code>	<code>0xf008</code>
<code>(%rdx, %rcx)</code>	<code>0xf000 + 0x0100</code>	<code>0xf100</code>
<code>(%rdx, %rcx, 4)</code>	<code>0xf000 + 0x0100*4</code>	<code>0xf400</code>
<code>0x80(%rdx, %rcx, 2)</code>	<code>0x80 + 0xf000 + 0x0100*2</code>	<code>0xf280</code>

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 3 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

